# Características de repositórios populares

Laboratório de Experimentação de Software

**Integrantes:** André Cota, Jhonatan Gutemberg

**Link do Repositório:** https://github.com/andre-cota/LaboratorioExperimentacoesSoftware

1. Contextualização

No cenário contemporâneo da Engenharia de Software, o GitHub transcendeu sua função original de hospedagem de código para se tornar o principal ecossistema de inovação open-sucre. Com milhões de repositórios ativos, a plataforma serve como um laboratório de dados massivos para entender tendências tecnológicas. No entanto, a alta visibilidade de um projeto (medida por "estrelas") nem sempre garante sua sustentabilidade técnica. Este estudo contextualiza a necessidade de métricas empíricas para diferenciar popularidade momentânea de maturidade e saúde operacional.

**Problema Foco do Experimento**

O problema central reside na identificação de padrões de sucesso e manutenção em sistemas de software de larga escala. Frequentemente, decisões de arquitetura e adoção de stacks tecnológicas são baseadas em tendências superficiais. O experimento foca em investigar se o volume de estrelas de um repositório é um indicador confiável de uma comunidade ativa, entregas frequentes e resolução eficiente de falhas, ou se reflete apenas o acúmulo histórico de visibilidade.

2. Escopo e Hipoteses

**Escopo de Pesquisa**

Questões-Problema (Research Questions)

As questões abaixo norteiam a coleta e análise dos dados:

RQ 01: Sistemas populares são necessariamente maduros (antigos)?

RQ 02: Sistemas populares recebem um volume proporcional de contribuição externa (Pull Requests)?

RQ 03: Sistemas populares mantêm uma cadência alta de lançamentos (Releases)?

RQ 04: Qual a frequência real de atualização (atividade recente) desses sistemas?

RQ 05: Quais as linguagens predominantes nos repositórios de alta adoção?

RQ 06: Sistemas populares possuem eficiência na gestão de falhas (Issues fechadas)?

**Hipóteses**

H1 (Maturidade): Sistemas populares são antigos, acumulando estrelas ao longo de vários anos.

H2 (Contribuição): A popularidade atrai uma comunidade ativa, resultando em um alto número de pull requests aceitos.

H3 (Manutenção): Projetos populares possuem um ciclo de vida vibrante, com releases frequentes.

H4 (Atualização): Projetos populares possuem uma maior interação por parte da comunidade fazendo com que tenham atualizações constantes.

H5 (Linguagens): A maioria dos sistemas é escrita em linguagens de alta adoção no mercado, como JavaScript, Python e TypeScript.

H6 (Qualidade/Issues): Grandes comunidades mantêm uma alta taxa de issues fechadas devido a processos robustos de triagem e alta quantidade de apoiadores.

H7 (Popularidade/Linguagem): O uso de linguagens difundidas facilita a contribuição, gerando mais atualizações e releases.

3. Objetivos

**Objetivo Principal**

Analisar quantitativamente o comportamento e a saúde dos 1.000 repositórios mais populares do GitHub através de extração via API GraphQL, correlacionando popularidade com métricas reais de manutenção e tecnologia.

**Objetivos Específicos**

Implementar um script de coleta resiliente capaz de minerar dados da API v4 do GitHub sem perda de performance.

Filtrar a amostra para garantir que apenas sistemas de software reais sejam analisados, excluindo repositórios de conteúdo que nao fazem sentido.

Calcular a taxa de eficiência de resolução de problemas (razão de issues) como indicador de qualidade de governança do projeto.

**Metodologia**

A metodologia aplicada seguiu uma abordagem estritamente quantitativa e experimental, fundamentada na extração de dados massivos (Data Mining) de repositórios reais. O objetivo foi analisar a correlação entre a popularidade de um software e sua saúde operacional,

utilizando o GitHub como fonte primária por ser o maior ecossistema de código aberto do mundo.

**Passo a Passo do Experimento**

1. Definição das Querys: Elaboração de consultas em GraphQL para extrair métricas específicas (estrelas, linguagens, datas e contagem de eventos).
2. Automação da Coleta: Execução de um script em Python que percorre a API v4 do GitHub, filtrando os 1.000 repositórios mais populares.
3. Tratamento de Dados (Sanitização): Aplicação do método is_software_system para excluir falsos positivos (listas "awesome", tutoriais e cursos), focando apenas em sistemas de software reais.
4. Armazenamento Incremental: Persistência dos dados em arquivo .csv a cada 100 registros para garantir a integridade dos dados e otimização de memória RAM.
5. Análise e Geração de Gráficos: Processamento do CSV através da classe analyze_repos para gerar as estatísticas descritivas e visualizações por questão de pesquisa (RQ).

**Decisões Técnicas**

- **Resiliência (Retry Logic)**: Implementação de um sistema de backoff incremental (timeout inicial de 15s com 3 tentativas) para lidar com instabilidades de rede ou limites de taxa da API (Rate Limiting).
- **Filtragem por Termos:** Decidiu-se pela exclusão rigorosa de termos como "tutorial" ou "course" para que a média de maturidade e contribuições não fosse distorcida por repositórios educacionais estáticos.

**Materiais Utilizados**

- Linguagem de Programação: Python
- Ambiente de Desenvolvimento: VS Code
- Interface de Dados: API v4 do GitHub (GraphQL).
- Bibliotecas de Suporte: requests (para chamadas HTTP), csv (para persistência) e bibliotecas de visualização de dados (Matplotlib/Seaborn) para a geração dos gráficos(Pandas).

**Métodos Utilizados**

Análise Estatística Descritiva: Para calcular médias de idade, percentuais de atualização e distribuição de linguagens.

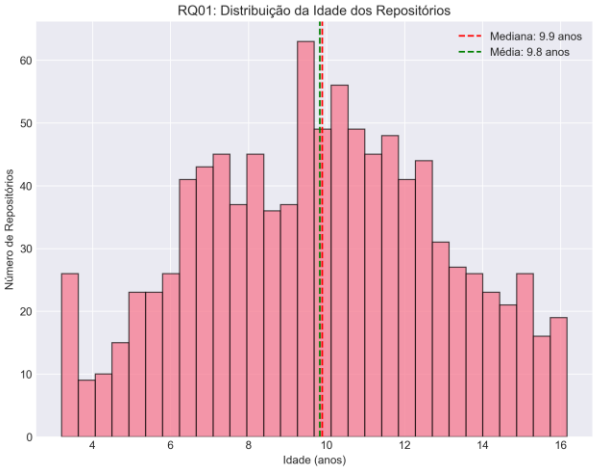
Filtragem Heurística: Uso de palavras-chave para identificar a natureza do repositório (Software vs. Conteúdo).

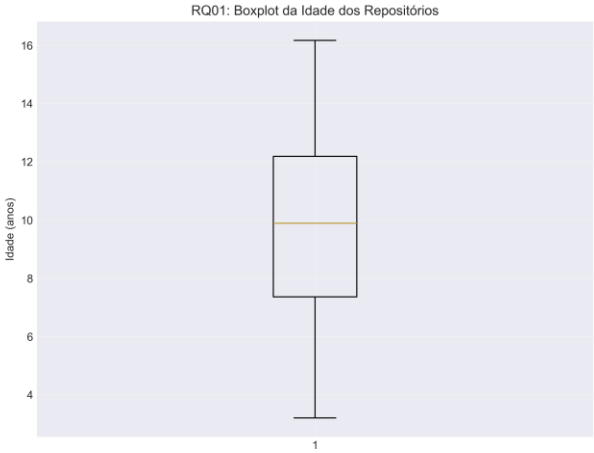
Cálculo de Razão (Ratio): Método aplicado especificamente na RQ 06 para determinar a eficiência de resolução de problemas (Issues fechadas / Total de Issues).

| Métrica | Unidade de Medida | Objetivo |
|---|---|---|
| **Idade do Repositório** | Anos (Calculado desde a criação) | Medir a maturidade (RQ 01) |
| **Pull Requests Aceitos** | Unidades (Contagem total) | Medir contribuição externa (RQ 02) |
| **Total de Releases** | Unidades (Contagem total) | Medir frequência de entregas (RQ 03) |
| **Tempo desde Atualização** | Dias/Semanas/Meses | Medir atividade recente (RQ 04) |
| **Linguagem Primária** | Categórica (String) | Identificar dominância tecnológica (RQ 05) |
| **Taxa de Fechamento** | Porcentagem (%) | Medir eficiência da comunidade (RQ 06) |

**Resultados Obtidos / Resultados Esperados**

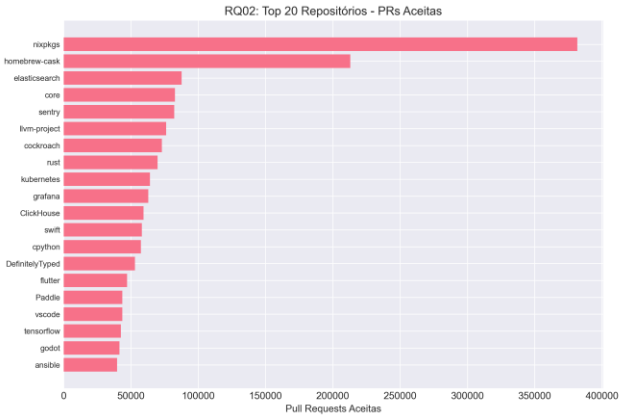
**RQ 01**

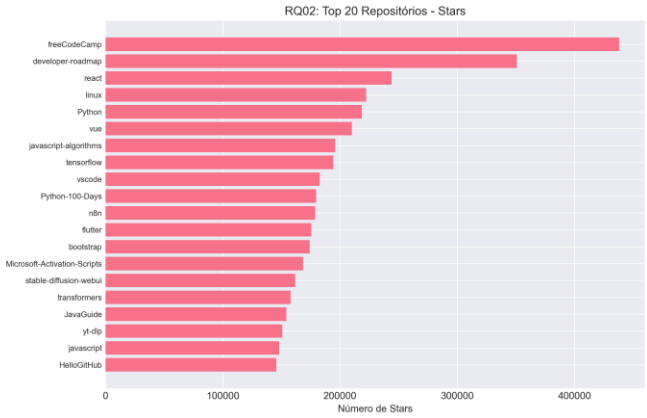



Esperávamos que os repositórios mais maduros tivessem uma idade mais próxima dos 20 anos, porém, em sua grande maioria, os repositórios com maior maturidade estão em volta dos 9,9 anos, praticamente metade do tempo esperado, ainda sim os repositórios mais maduros são um pouco mais velhos.

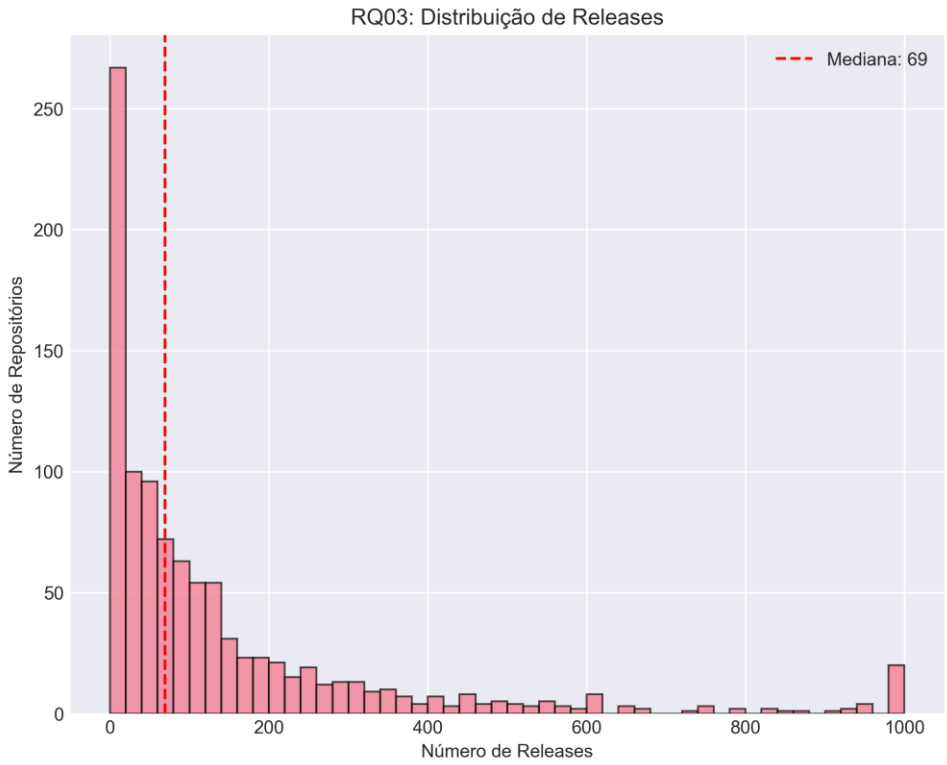
**RQ 02**



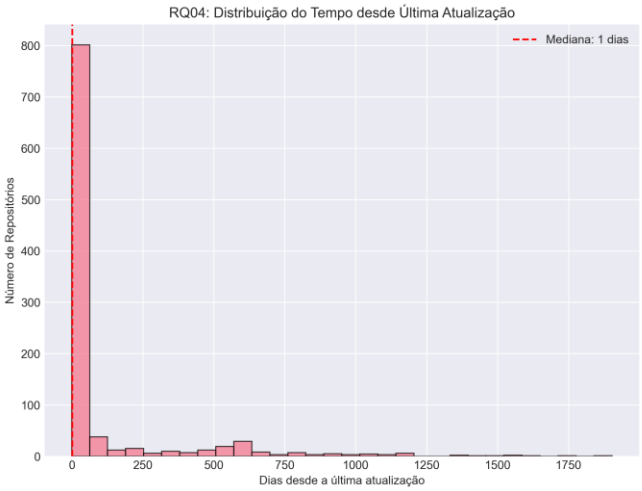

Esperávamos que os repositórios mais populares tivessem a maior quantidade de pull requests aceitas já que quanto mais pull request, maiores seriam as interações da comunidade, porém baseando-se na quantidade de estrelas os repositórios possuem, encontramos um outro resultado.

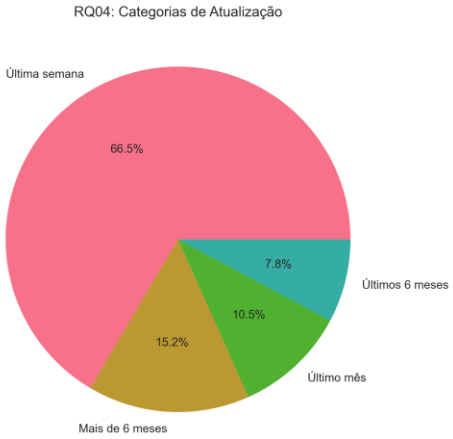
**RQ 03**




Esperávamos que os repositórios mais populares tivessem uma frequência mais alta de releases, o que entendemos que seja o que realmente acontece, já que os 20 repositórios mais populares possuem cada um mais de 1000 release.

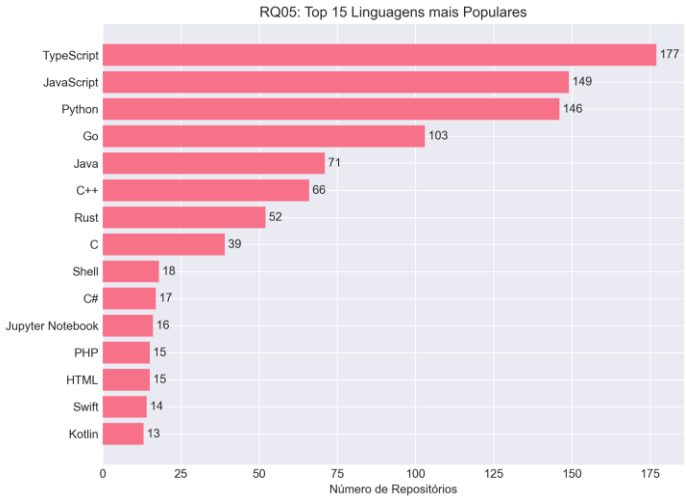
**RQ 04**

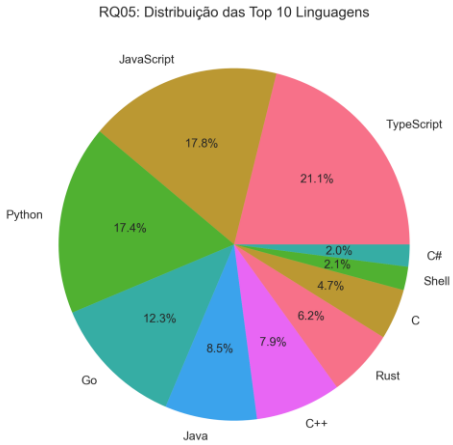



O que imaginávamos que os sistemas mais populares eram atualizados com frequência e como podemos observar é que realmente isso ocorre, visto que dos 1000 repositórios analisados, 77% deles foram atualizados até o último mês, sendo que 66,5% do total foi atualizado até a última semana.

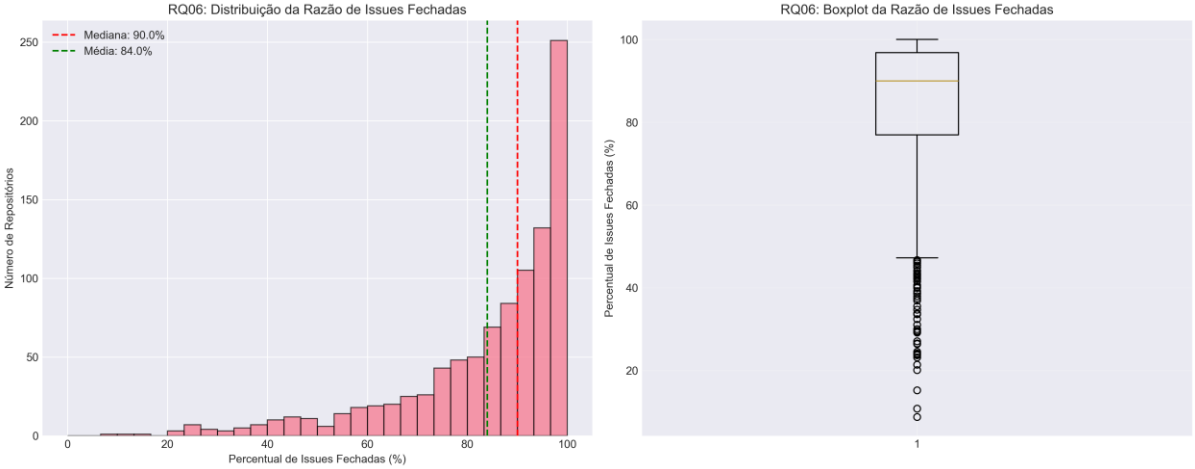
## RQ 05



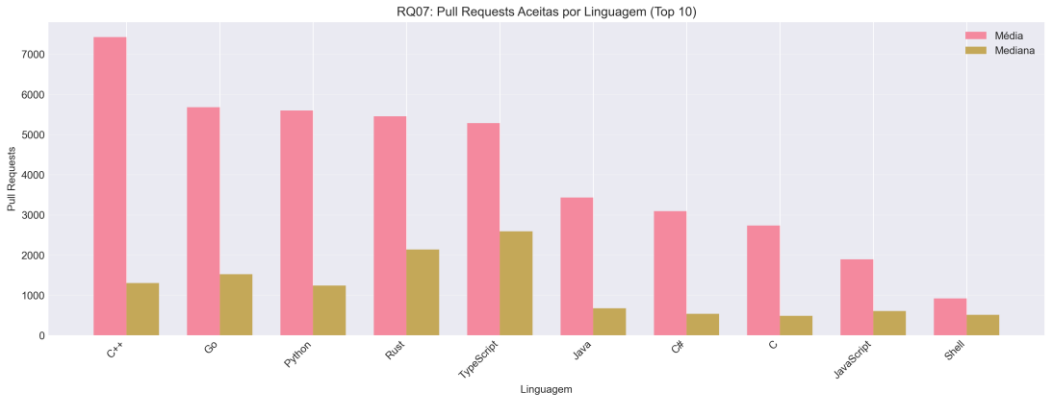

Nossa hipótese é que as linguagens mais comuns seriam as com mais repositórios, o que se confirmou, visto que o top 5 dos repositórios baseando na linguagem primária são consequentemente: TypeScript(177), JavaScript(149), Python(146), Go(103), Java(71).

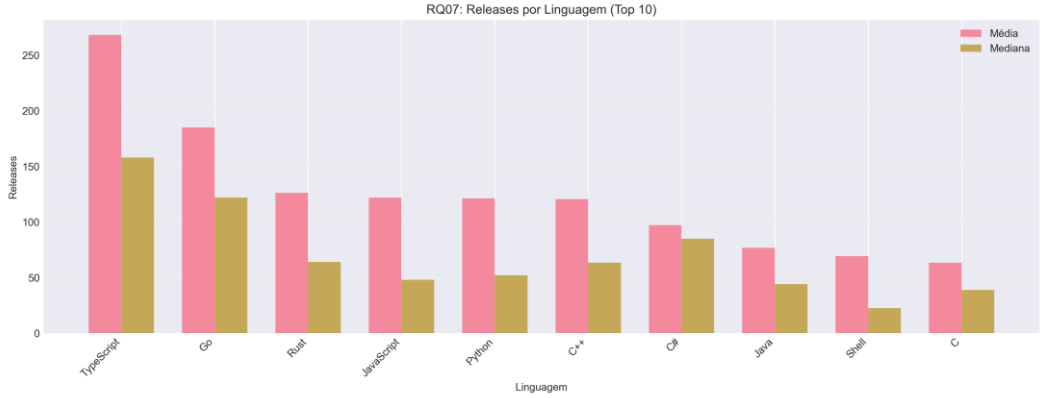
**RQ 06**

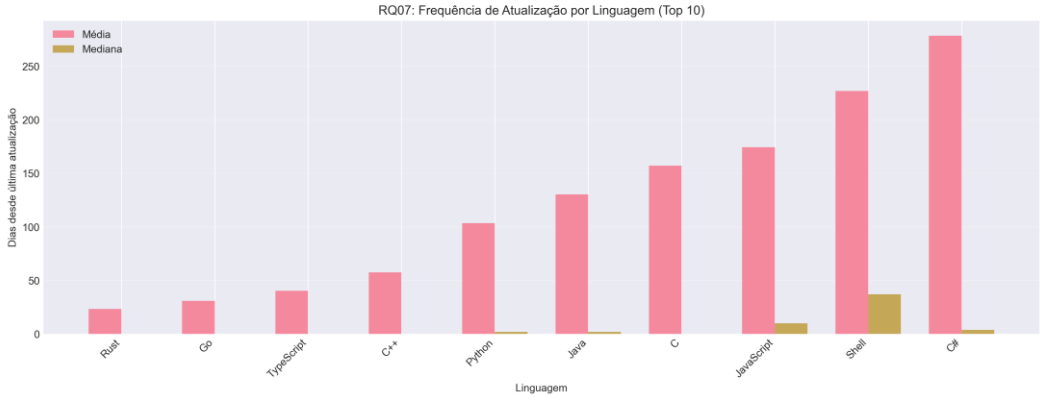



O que pensamos é que sim, os repositórios possuiriam a maior quantidade de issues fechadas, visto que são sistemas robustos e com uma grande quantidade de participantes, o que se concretizou, visto que mais 80% das issues estão fechadas.

**RQ 07**




Tínhamos em mente que os sistemas que escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência e é basicamente o que ocorre, não necessariamente todos na mesma ordem mas sim é o que ocorre.

**Dados interessantes:**

• Total de stars: 43,663,403

• Total de PRs aceitas: 4,997,999

• Total de releases: 143,163

• Total de issues: 5,659,904

• Linguagens diferentes: 49

**Conclusão:**

A análise dos 1.000 repositórios extraídos via API v4 do GitHub confirma que a popularidade de um ecossistema de software está diretamente ligada à sua vivacidade operacional. Observou-se que sistemas com alta adoção não são apenas "bibliotecas famosas", mas organismos ativos: 77% dos projetos foram atualizados no último mês e a taxa de resolução de issues superou 80%. O estudo revelou que a maturidade de um projeto de topo se consolida em torno dos 9,9 anos, desmistificando a ideia de que apenas projetos com décadas de existência dominam o cenário open-source.

**Tomada de decisão**

Para desenvolvedores e gestores de projetos que buscam sustentabilidade e suporte comunitário, a escolha por TypeScript, JavaScript ou Python apresenta-se como a decisão mais estratégica. Os dados mostram que essas linguagens garantem uma barreira de entrada menor para contribuidores, resultando em um ciclo de releases mais curto (frequentemente acima de 1.000 releases por projeto) e uma resposta mais ágil a falhas críticas. Optar por essas tecnologias reduz o risco de "estagnação" do código a médio prazo.

**Sugestões futuras**

Para trabalhos subsequentes, recomenda-se:

Análise Qualitativa de Commits: Investigar se o alto volume de pull requests em linguagens populares está relacionado a correções de sintaxe ou a novas funcionalidades complexas.

Estudo de Correlação Star vs. Issue: Avaliar se o aumento súbito de "estrelas" impacta negativamente o tempo de resposta das issues (sobrecarga da equipe).

Métricas de Diversidade: Analisar a distribuição geográfica dos contribuidores para entender o alcance global de cada linguagem.

**Resultado conclusivo sucinto**

Os sistemas de software mais bem-sucedidos do GitHub são escritos em linguagens de alta penetração de mercado (TypeScript/JavaScript), possuem um ciclo de vida de aproximadamente 10 anos e mantêm um fluxo de manutenção ininterrupto, validando a premissa de que a popularidade atrai e sustenta comunidades eficientes.

**Plus: Confronto com a Literatura Científica**

Os resultados obtidos neste laboratório corroboram diversas evidências da literatura recente:

Corroboração (StackOverflow 2025): Seus dados de que TypeScript (177) e JavaScript (149) lideram o volume de projetos populares estão em total harmonia com o Developer Survey 2025, que aponta estas linguagens como as de maior uso profissional e desejo de aprendizado.